Object Oriented Programming with C++

ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment : Individual
Duration : 1 Week
Submission Date : _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192524021simats-lang/DSA0105-OOPS

I have your uploaded **Namma Metro Smart-Card Fare System – C++ Assignment (Day 1 to Day 4)**.

I'll provide the answers in simple, beginner-friendly C++, following all 12 tasks, including:

- Detailed explanations for theory questions.
- Pseudocode and C++ programs.
- Predicted outputs and actual outputs.
- Test cases with expected results.
- Alternative concepts and time/space complexity.
- Final Day-4 design challenge with revised code.

TASK 1 – Understand the Problem and Identify the Objects

1. Objects in the metro fare system

Object	Data (Attributes)	Behaviour (Methods)
SmartCard	Card ID, balance	Recharge, deduct fare, check balance
Passenger	Name, passenger type	Travel, receive concession
Journey	Source zone, destination zone, time	Calculate fare
Zone	Zone name, base fare	Return zone fare
FareTable	Zone fares, surcharge	Calculate fare

Explanation:

- SmartCard stores the passenger's money and ensures the balance does not fall below Rs 50.
- Passenger stores passenger details and identifies students and senior citizens.
- Journey stores travel information and calculates the applicable fare.
- Zone represents Inner, Middle and Outer zones.
- FareTable stores all fare values in one place, making fare changes easier.

2. Procedural vs Object-Oriented Programming

Procedural	Object-Oriented
Uses functions and variables	Uses classes and objects
Data is generally separate	Data and functions are combined
Less data protection	Supports data protection
Changes may affect many functions	Changes can be isolated
Example: calculateFare()	Example: FareTable.calculateFare()

3. Four principles of OOP

1. **Encapsulation:** Keeping the smart-card balance private and allowing access through methods.
2. **Abstraction:** Showing only necessary functions such as recharge() and deduct().
3. **Inheritance:** Student and SeniorCitizen can inherit common properties from Passenger.
4. **Polymorphism:** Different passenger types can implement their own concession calculation.

4. Rules of a valid journey

- The fare must be based on the farthest zone travelled.
- Peak surcharge applies from 8 AM to before 10 AM and from 5 PM to before 8 PM.
- Student and senior citizen concessions must be applied correctly.
- The card must have enough money to pay the fare.
- The remaining balance must be at least Rs 50.
- Negative balances are not allowed.
- All calculations must be exact.

5. Alternative concept and complexity analysis

If procedural programming is used, global variables can store fares and balances, while separate functions calculate fares and deduct money.

Factor	Procedural	OOP
Time per journey	O(1)	O(1)
Memory per card	O(1)	O(1)
Fare rule modification	May affect multiple functions	Usually changes FareTable
Data protection	Less	Better

Conclusion: OOP is better because it provides data protection, code reusability and easier maintenance.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

1. Five types of tokens in C++

Token	Example	Meaning
Keywords	int, class, if, return	Reserved words
Identifiers	balance, fare, SmartCard	User-defined names
Constants	1000, 5000, 20	Fixed values
Operators	+, -, *, /, ==	Perform operations

Punctuators ;, , (), ,	Separate program elements	

2. #define, const and constexpr

- `#define`: Defines a macro without a data type.
- `const`: Creates a value that cannot be modified.
- `constexpr`: Creates a compile-time constant when its initializer permits it.

Why constexpr is preferred: It provides type checking and allows values to be used in compile-time expressions.

3. Constants for the metro system

```
None
constexpr int INNER_FARE = 1000;

constexpr int MIDDLE_FARE = 2000;

constexpr int OUTER_FARE = 3000;


constexpr int PEAK_SURCHARGE = 20;

constexpr long long MIN_BALANCE = 5000;
```

All money values are stored in paise.

4. enum and enum class

```
None
enum class Zone {
    Inner,
    Middle,
    Outer
};
```

`enum class` is safer because its values are scoped and cannot be automatically converted to integers.

5. Naming conventions

- Use meaningful names such as `totalFare`.
- Use uppercase letters for constants.
- Use camelCase for variables and functions.
- Do not use spaces or special symbols in identifiers.
- Do not use C++ keywords as identifiers.

Alternative concept and complexity analysis

If `#define` is used instead of `constexpr`:

Feature	#define	constexpr
Type checking	No	Yes
Scope	Preprocessor-based	C++ scope
Memory	Usually no storage	Usually no storage
Execution time	No significant difference	No significant difference
Debugging	More difficult	Easier

Conclusion: `constexpr` is preferred because it provides safer and more readable code.

TASK 3 – Choose Data Types and Check Type Compatibility

1. Data type selection

Value	Data type	Reason
Fare	long long	Stores paise exactly
Balance	long long	Prevents overflow
Time	int	Stores hour
Zone number	enum class	Represents zones safely
Journey count	long long	Handles millions of journeys
Peak status	bool	True or false
Passenger initial	char	Stores one character
Average fare	double	Stores calculated averages

2. Why use integer paise instead of double?

Money should be stored in integer paise because floating-point numbers can produce rounding errors.

Example:

```
None
#include <iostream>

using namespace std;


int main() {
```

```
    double a = 0.1;
    double b = 0.2;

    cout << a + b;
    return 0;
}
```

Possible output:

```
None
0.3
```

The internal result may not be exactly 0.3.

Correct method:

```
None
#include <iostream>
using namespace std;

int main() {
    long long a = 10;
    long long b = 20;

    cout << a + b << " paise";
    return 0;
}
```

Output:

```
None
30 paise
```

3. Type compatibility

```
None
int a = 5;

double b = 2.5;

bool c = 1;


double result = a + b;
```

- `int` is converted to `double` during addition.
- A non-zero number converts to `true`.
- Zero converts to `false`.

4. Size and range of data types

Typical sizes on a modern 64-bit system:

Data type	Size	Approximate range
bool	1 byte	false or true
char	1 byte	-128 to 127 (signed char)
int	4 bytes	-2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	0 to 4,294,967,295
long long	8 bytes	Approximately $\pm 9.22 \times 10^{18}$

float	4 bytes	About 6–7 decimal digits
double	8 bytes	About 15–16 decimal digits

Actual sizes should be checked using `sizeof()` and limits using `<climits>`.

5. Is int enough for network collection?

No.

For 3,000,000 journeys at Rs 36 each:

$$3,000,000 \times 3600 = 10,800,000,000 paise$$

This exceeds the maximum signed 32-bit integer value.

Therefore, `long long` is required.

Alternative concept and complexity analysis

Factor	double rupees	long long paise
Addition time	O(1)	O(1)
Memory	Usually 8 bytes	8 bytes
Precision	Approximate	Exact for whole paise
Overflow	Very large range	Finite integer range

Conclusion: Use `long long` for money and totals.

TASK 4 – Control Implicit Conversions

1. Implicit conversions

Implicit conversion happens automatically when C++ changes one data type into another.

Examples:

- Integer division.
- Integer to double conversion.
- Character to integer promotion.
- Integer to Boolean conversion.
- Signed to unsigned conversion.

2. Predict the output

Given program:

```
None
#include <iostream>
using namespace std;

int main() {
    int a = 7, b = 2;

    double c = a / b;
    double d = static_cast<double>(a) / b;

    cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    cout << ch << ' ' << ch + 1 << '\n';
```

```
    bool flag = 5;
    cout << flag << ' ' << boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;

    cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Predicted output:

```
None
3 3.5
C 68
1 true
s >= u
```

3. Explanation

Line 1:

```
None
double c = a / b;
```

Integer division occurs first.

7 / 2 = 3

Then 3 converts to 3.0.

```
None
double d = static_cast<double>(a) / b;
```

Here, 7 converts to double before division.

7 / 2 = 3.5

Line 2:

```
None
char ch = 'A' + 2;
```

A has ASCII value 65.

65 + 2 = 67, which represents C.

`ch + 1` promotes the character to int and produces 68.

Line 3:

```
None
bool flag = 5;
```

Any non-zero number converts to true.

- `1` is the default Boolean output.
- `true` is printed using `boolalpha`.

Line 4:

```
None
s < u
```

The signed value -1 is converted to unsigned. It becomes a very large unsigned value.

Therefore, the condition is false.

4. Alternative concept

Using explicit conversions:

```
None
double result = static_cast<double>(a) / b;

int number{5};
```

Brace initialization prevents unsafe narrowing conversions.

For example:

```
None
int x{3.7}; // Compilation error
```

Factor	Implicit conversion	Explicit conversion
Time	O(1)	O(1)
Memory	Same	Same
Safety	Less clear	More controlled
Errors	Some remain unnoticed	Some caught at compile time

Conclusion: Explicit conversion improves readability and prevents unexpected results.

TASK 5 – Write Correct Expressions with the Right Precedence

1. Peak-hour expression

```
None
(hour >= 8 && hour < 10) ||
```

```
(hour >= 17 && hour < 20)
```

Peak hours:

- 8:00 AM to before 10:00 AM.
- 5:00 PM to before 8:00 PM.

10 AM and 8 PM are not peak hours.

2. Surcharge expression

```
None
fare = peak ? fare * 120 / 100 : fare;
```

The fare increases by 20% during peak hours.

3. Concession expression

The assignment does not specify the concession percentage. For the example, assume a 10% concession:

```
None
fare = concession ? fare * 90 / 100 : fare;
```

This is an assumption and can be changed when the actual concession rate is provided.

4. Increment operators

Operator	Meaning
x++	Uses value first, then increments
++x	Increments first, then uses value

`i = i++` should be avoided because it creates confusing and potentially undefined behaviour in older C++ standards; in modern C++, it has a defined result that often surprises beginners.

5. Division and modulus with negative values

```
None
-10 / 3 = -3

-10 % 3 = -1
```

C++ integer division truncates towards zero.

6. Predict the output

```
None
#include <iostream>
using namespace std;

int main() {
    int x = 5, y = 3;

    cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    cout << z << ' ' << x << ' ' << y << '\n';

    cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

    cout << 10 / 3 << ' ' << 10 % 3 << ' '
         << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
```

```
    for (int i = 0; i < 10; i += 3)
        k += i;

    cout << k << '\n';
}
```

Expected output:

```
None
11 16
9 6 4
6 4
3 1 -3 -1
18
```

7. Explanation

1. `x + y * 2`: multiplication happens first. Result = 11.
2. `(x + y) * 2`: brackets are evaluated first. Result = 16.
3. `x++ + ++y`: old x = 5, new y = 4. Result = 9. Then x = 6.
4. `x > y ? x : y`: selects 6.
5. `x % 4 * 2`: modulus first, then multiplication. Result = 4.
6. Integer division and modulus produce 3, 1, -3 and -1.
7. Loop values are 0, 3, 6 and 9. Total = 18.

8. Alternative concept

A lookup table can be used:

```
None
bool peak[24] = {};
```

```
for (int i = 8; i < 10; i++)
    peak[i] = true;

for (int i = 17; i < 20; i++)
    peak[i] = true;
```

Check:

```
None
if (peak[hour])
    cout << "Peak hour";
```

Factor	Logical expression	Lookup table
Time	O(1)	O(1)
Extra memory	O(1)	24 Boolean elements
Readability	Good	Good
Comparisons	Multiple	Direct lookup

Conclusion: Both methods work. The logical expression is simpler for only two time periods.

TASK 6 – Design the Control Structures

1. Control structures

- `if-else`: Checks peak hours and minimum balance.
- `switch`: Selects the fare according to the zone.
- Conditional operator: Selects between two values.
- `for`: Processes a known number of journeys.
- Range-based `for`: Processes all journey records.
- `while`: Processes journeys until input ends.
- `break`: Exits a loop or switch.
- `continue`: Skips an invalid journey.

2. Pseudocode

```
None
START

DECLARE balance = 10000 paise
DECLARE minimum balance = 5000 paise

INPUT zone
INPUT hour

IF hour is between 8 and 10 OR 17 and 20
    SET peak = TRUE
ELSE
    SET peak = FALSE
END IF

SWITCH zone

    CASE INNER:
        SET fare = 1000
```

```
        BREAK

    CASE MIDDLE:
        SET fare = 2000
        BREAK

    CASE OUTER:
        SET fare = 3000
        BREAK

    DEFAULT:
        DISPLAY "Invalid zone"
        STOP
END SWITCH

IF peak is TRUE
    SET fare = fare + (fare * 20 / 100)
END IF

IF balance - fare >= minimum balance
    balance = balance - fare
    DISPLAY fare and balance
ELSE
    DISPLAY "Journey denied"
END IF
```

```
STOP
```

3. Complete C++ program

```
None

#include <iostream>
#include <iomanip>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak) {
    int base = 0;

    switch (z) {
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;

        case Zone::Middle:
```

```
            base = BASE_PAISE[1];
            break;

        case Zone::Outer:
            base = BASE_PAISE[2];
            break;

        default:
            return -1;
    }

    if (peak)
        base = base + base * PEAK_SURCHARGE_PERCENT / 100;

    return base;
}

bool canTravel(long long balance, long long f) {
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main() {
    long long balance = 10000;

    struct Trip {
```

```
    Zone zone;
    bool peak;
};

Trip trips[] = {
    {Zone::Inner, false},
    {Zone::Outer, true},
    {Zone::Middle, false}
};

for (const Trip& t : trips) {
    long long f = fare(t.zone, t.peak);

    if (canTravel(balance, f)) {
        balance -= f;

        cout << "Fare Rs "
             << f / 100 << '.'
             << setw(2) << setfill('0') << f % 100
             << setfill(' ');
    } else {
        cout << "Denied (low balance)";
    }

    cout << " balance Rs " << balance / 100 << '\n';
```

```
    }

    return 0;
}
```

4. Expected output

```
None
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

5. Boundary testing

Time	Peak or Off-peak
8:00 AM	Peak
10:00 AM	Off-peak
5:00 PM	Peak
8:00 PM	Off-peak

6. Alternative concept

An array can replace the switch:

```
None
constexpr long long FARE[] = {1000, 2000, 3000};
```

Factor	Switch	Array
Lookup time	O(1)	O(1)
Extra storage	No fare array required	Stores fare values
Fourth zone	Add another case	Add another array element
Flexibility	Good	Better for table-driven design

Conclusion: The switch is suitable for three zones. An array becomes easier to maintain when more zones are added.

TASK 7 – From Procedural to Object-Oriented

1. Procedural programming

In procedural programming, balance and fare are stored separately, and functions perform the operations.

```
None
#include <iostream>
using namespace std;

long long balance = 10000;
const long long MIN_BALANCE = 5000;
```

```
bool canTravel(long long fare) {
    return balance - fare >= MIN_BALANCE;
}

void deduct(long long fare) {
    if (canTravel(fare))
        balance -= fare;
    else
        cout << "Journey denied\n";
}

int main() {
    deduct(1000);
    cout << "Balance: " << balance << " paise";
    return 0;
}
```

Output:

```
None
Balance: 9000 paise
```

2. Object-oriented programming

```
None
#include <iostream>
using namespace std;
```

```
class SmartCard {
private:
    long long balance;
    const long long MIN_BALANCE = 5000;

public:
    SmartCard(long long b) {
        balance = b;
    }

    bool canTravel(long long fare) {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    void deduct(long long fare) {
        if (canTravel(fare)) {
            balance -= fare;
            cout << "Payment successful\n";
        } else {
            cout << "Journey denied\n";
        }
    }

    long long getBalance() {
```

```
        return balance;
    }
};

int main() {
    SmartCard card(10000);

    card.deduct(1000);

    cout << "Balance: "
         << card.getBalance() << " paise";

    return 0;
}
```

Output:

```
None
Payment successful
Balance: 9000 paise
```

3. Comparison

Procedural	OOP
Balance is stored separately	Balance is private
Functions access shared data	Methods control access

Negative balance can be introduced by other code	Controlled methods prevent invalid deductions
Changes may affect multiple functions	Changes can be isolated

4. Alternative concept

If balance is public:

```
None
class SmartCard {
public:
    long long balance;
};
```

Any part of the program can modify it.

For example:

```
None
card.balance = -500;
```

This breaks the rule that the card balance must never become negative.

Conclusion: Private data members and public methods are better because they protect the balance. Time and memory complexity remain unchanged.

TASK 8 – Read and Print Data Safely

1. Reading input using cin

```
None
int hour;
cin >> hour;
```

```
if (cin.fail()) {
    cout << "Invalid input";
}
```

`cin.fail()` checks whether the input operation failed.

2. Clearing invalid input

```
None
cin.clear();
cin.ignore(1000, '\n');
```

- `cin.clear()` resets the error state.
- `cin.ignore()` removes the invalid input.

3. Formatting money

```
None
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double amount = 25.50;

    cout << fixed << setprecision(2);
    cout << "Amount: Rs " << amount;
```

```
    return 0;
}
```

Output:

```
None
Amount: Rs 25.50
```

4. Safe input program

```
None
#include <iostream>
using namespace std;

int main() {
    int hour;

    cout << "Enter hour: ";

    while (!(cin >> hour)) {
        cout << "Invalid input. Enter a number: ";

        cin.clear();
        cin.ignore(1000, '\n');
    }

    if (hour < 0 || hour > 23) {
        cout << "Invalid hour";
```

```
    } else {
        cout << "Valid hour";
    }

    return 0;
}
```

5. Alternative concept – getline() and stoi()

None

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string input;

    cout << "Enter hour: ";
    getline(cin, input);

    try {
        size_t pos;
        int hour = stoi(input, &pos);

        if (pos != input.size())
            throw invalid_argument("Invalid");
```

```
        cout << "Hour: " << hour;
    }
    catch (...) {
        cout << "Invalid input";
    }

    return 0;
}
```

Factor	cin	getline + stoi
Processing time	O(L)	O(L)
Memory	Small input buffer	Stores input string
Invalid input	clear and ignore	try and catch
Flexibility	Reads formatted values	Reads complete lines

Conclusion: Both methods are useful. `getline()` with `stoi()` provides better control over complete input lines.

TASK 9 – Implement and Test the Fare Logic

1. Testing table

Assume the following rules:

- Inner = Rs 10
- Middle = Rs 20
- Outer = Rs 30
- Peak surcharge = 20%
- Minimum remaining balance = Rs 50

Test	Zone	Time	Expected fare
1	Inner	11 AM	Rs 10
2	Middle	6 PM	Rs 24
3	Outer	12 PM	Rs 30
4	Outer	9 AM	Rs 36
5	Inner	8 AM	Rs 12
6	Inner	10 AM	Rs 10
7	Middle	8 PM	Rs 20
8	Middle	5 PM	Rs 24

2. Balance boundary test

Initial balance = Rs 60.

Fare = Rs 10.

Remaining balance:

$$60 - 10 = 50$$

Journey allowed

If the balance is Rs 59:

$$59 - 10 = 49$$

Journey denied

3. Invalid test cases

Input	Expected result
Unknown zone	Invalid zone
Negative recharge	Reject recharge
Letters for hour	Invalid input
Hour = 25	Invalid hour
Negative fare	Reject fare

4. Five-journey manual calculation

Assume the passenger has Rs 200.

Journey	Fare
Inner off-peak	Rs 10

Middle peak	Rs 24
Outer off-peak	Rs 30
Inner peak	Rs 12
Outer peak	Rs 36
Total	**Rs 112**

Remaining balance:

$$200 - 112 = 88$$

Final balance = Rs 88.

5. Alternative concept

Instead of writing separate tests, a test-case table can be used.

```
None
struct Test {
    int zone;
    bool peak;
    long long expected;
};

Test tests[] = {
    {0, false, 1000},
    {1, true, 2400},
```

```
    {2, false, 3000},
    {0, true, 1200}
};
```

A loop can compare the actual fare with the expected fare.

Factor	Separate tests	Test table
Time	O(t)	O(t)
Memory	Small	O(t)
Adding new tests	More code	Add one row
Maintenance	Difficult	Easier

Conclusion: Table-driven testing is better because new test cases can be added easily.

TASK 10 – Compare Expectations with Execution

1. Journey growth

The metro expects the following journey counts:

Network	Journeys per day
One station	20,000

One line	500,000
Whole network	3,000,000

For comparison, assume every journey costs Rs 36 (3,600 paise).

2. Total collection

Journeys	Total in rupees	Total in paise
20,000	Rs 7,20,000	72,000,000
500,000	Rs 1,80,00,000	1,800,000,000
3,000,000	Rs 10,80,00,000	10,800,000,000

3. Integer overflow

Maximum signed 32-bit integer:

$$2,147,483,647$$

The whole network total is:

$$10,800,000,000$$

Therefore, `int` cannot store the complete amount in paise.

Use:

```
None
long long total = 0;
```

4. Integer paise vs double rupees

- Integer paise stores exact whole-paise amounts.
- Double stores approximate decimal values.
- Both usually use 8 bytes when using `long long` and `double`.
- Both perform addition in O(1) time.

The accumulated floating-point error depends on the values and number of additions. It is not necessarily a fixed amount per journey.

5. Alternative concept

Data type	Result for 3 million journeys
int	Overflow
long long	Correct exact integer total
double	Large enough, but decimal precision is approximate

Compiler warnings:

Compile using:

```
None
g++ -Wall -Wextra metro.cpp -o metro
```

Possible warnings include:

- Signed/unsigned comparison.
- Unused variables.
- Possible narrowing conversions.
- Missing return statements.

Actual warnings depend on the code and compiler. They should be reported after compiling the final program, not assumed in advance.

Conclusion: `long long` is the recommended data type for total fare collection.

TASK 11 – Recommend and Defend the Final Design

1. Recommended data types

Data	Recommended type
Fare	long long
Balance	long long
Zone	enum class
Peak status	bool
Hour	int
Journey count	long long
Fare constants	constexpr

2. Recommended classes

1. SmartCard – manages balance.
2. Passenger – stores passenger details.
3. Journey – stores journey information.
4. FareTable – manages fare calculations.

3. How to change a fare

For example, if Inner Zone fare changes from Rs 10 to Rs 15:

```
None
constexpr int INNER_FARE = 1500;
```

The fare needs to be changed in only one place.

4. Limitations

- Adding more zones requires updating the fare table.
- New concessions require additional rules.
- Monthly passes need a separate class.
- Input validation is necessary.
- Large network operations need performance testing.

5. Does the design support the whole network?

Yes, with suitable data types and efficient processing.

The design uses:

- `long long` for large collections.
- O(1) fare lookup.
- O(1) balance checking.
- Reusable classes.
- Exact integer-paise calculations.

6. Alternative concept – Configuration file

Fare details can be stored in a configuration file and read when the metro gate starts.

Advantages:

- Fare changes do not require recompilation.
- Easier maintenance.
- New zones can be added more conveniently.

Disadvantages:

- File-reading time is required at startup.
- Invalid configuration data must be checked.
- The program needs memory to store the fare table.

If there are z zones, reading the configuration takes O(z) time and storing the fares requires O(z) space.

Final recommendation: Use an object-oriented design with `constexpr` constants initially. For a growing metro network, use a validated configurable fare table.

TASK 12 – Re-evaluate and Adapt the Design (DAY 4)

8 Marks

A. Impact analysis – 2 Marks

The metro introduces three new requirements:

1. Fourth zone – Airport, fare Rs 60.
2. Weekend discount – 10%.
3. Monthly pass – Unlimited journeys in two selected zones.

Changes required:

Original design	New design
Three zones	Four zones
Three fare constants	Four fare constants
Peak surcharge only	Peak surcharge and weekend discount
SmartCard class	SmartCard + MonthlyPass
Switch statement	Fare table indexed by zone

B. Evaluation of original design – 2 Marks

1. Switch-based fare logic

The original switch supports three zones. The Airport zone requires another case.

```
None
case Zone::Airport:
```

```
    base = 6000;
    break;
```

This works, but adding more zones requires modifying the switch.

2. SmartCard class

The SmartCard class can still be retained because it already manages balance and deductions.

However, monthly-pass information should be managed separately using a MonthlyPass class.

Worked examples

Journey	Calculation	Final fare
Inner, normal weekday	Rs 10	Rs 10
Outer, peak weekday	30 + 20%	Rs 36
Airport, normal weekday	Rs 60	Rs 60
Inner, weekend	10 - 10%	Rs 9
Outer, weekend peak	Weekend discount only	Rs 27
Monthly-pass covered zone	Unlimited pass	Rs 0

The weekend discount does not combine with the peak surcharge.

C. Adaptation or redesign – 2 Marks

Recommended: Modify the original design

We can retain SmartCard but modify the fare table and introduce a MonthlyPass class.

Complete revised C++ program

None

```
#include <iostream>
using namespace std;

enum class Zone {
    Inner, Middle, Outer, Airport
};

constexpr long long FARE[] = {
    1000, 2000, 3000, 6000
};

class MonthlyPass {
private:
    bool covered[4];

public:
    MonthlyPass(int zone1, int zone2) {
        for (int i = 0; i < 4; i++)
            covered[i] = false;

        if (zone1 >= 0 && zone1 < 4)
            covered[zone1] = true;

        if (zone2 >= 0 && zone2 < 4)
```

```
            covered[zone2] = true;
    }

    bool hasPass(Zone z) const {
        int index = static_cast<int>(z);

        return index >= 0 && index < 4
               && covered[index];
    }
};

class SmartCard {
private:
    long long balance;
    static constexpr long long MIN_BALANCE = 5000;

public:
    SmartCard(long long b) : balance(b) {}

    bool canTravel(long long fare) const {
        return fare >= 0 &&
               balance - fare >= MIN_BALANCE;
    }

    void deduct(long long fare) {
```

```
        if (canTravel(fare)) {
            balance -= fare;
            cout << "Journey successful\n";
        } else {
            cout << "Journey denied\n";
        }
    }

    long long getBalance() const {
        return balance;
    }
};

long long calculateFare(
    Zone z, bool peak, bool weekend,
    const MonthlyPass& pass) {

    int index = static_cast<int>(z);

    if (index < 0 || index >= 4)
        return -1;

    // Monthly pass gives free travel
    if (pass.hasPass(z))
        return 0;
```

```
    long long fare = FARE[index];

    // Weekend discount does not combine
    // with peak surcharge
    if (weekend) {
        fare = fare * 90 / 100;
    }
    else if (peak) {
        fare = fare * 120 / 100;
    }

    return fare;
}

int main() {

    SmartCard card(20000);  // Rs 200

    // Monthly pass for Inner and Middle zones
    MonthlyPass pass(0, 1);

    struct Trip {
        Zone zone;
        bool peak;
```

```
    bool weekend;
};

Trip trips[] = {
    {Zone::Inner, false, false},
    {Zone::Outer, true, false},
    {Zone::Airport, false, true},
    {Zone::Outer, true, true}
};

for (const Trip& t : trips) {

    long long fare = calculateFare(
        t.zone, t.peak, t.weekend, pass
    );

    if (fare < 0) {
        cout << "Invalid zone\n";
        continue;
    }

    cout << "Fare: Rs "
         << fare / 100 << "."
         << (fare % 100 < 10 ? "0" : "")
         << fare % 100 << '\n';
```

```
        if (card.canTravel(fare)) {
            card.deduct(fare);
        } else {
            cout << "Insufficient balance\n";
        }

        cout << "Balance: Rs "
             << card.getBalance() / 100 << "\n\n";
    }

    return 0;
}
```

Expected output

```
None
Fare: Rs 0.00
Journey successful
Balance: Rs 200

Fare: Rs 36.00
Journey successful
Balance: Rs 164

Fare: Rs 54.00
```

```
Journey successful
Balance: Rs 110

Fare: Rs 27.00
Journey successful
Balance: Rs 83
```

Assumptions: The monthly pass covers two selected zones and gives free travel in those zones. The weekend discount replaces the peak surcharge. A pass-covered journey does not deduct money, so the minimum-balance condition is checked only for paid journeys.

D. Evidence-based defence – 2 Marks

Test case	Expected result	Program result
Inner with monthly pass	Rs 0	Rs 0
Outer peak	Rs 36	Rs 36
Airport weekend	Rs 54	Rs 54
Outer weekend peak	Rs 27	Rs 27

Advantages of modified design:

1. Supports four zones.
2. Supports weekend discounts.
3. Supports monthly passes.
4. Uses integer paise for exact calculations.
5. Protects the smart-card balance.
6. Makes future modifications easier.

Limitations:

- More conditions are required.
- Monthly-pass expiry and renewal are not implemented.
- Only two zones are supported per pass.
- Student and senior citizen concessions need additional rules.

Final conclusion

The original design should be modified rather than completely redesigned. The SmartCard class can be retained, while the fare table is expanded to four zones and a separate MonthlyPass class is introduced. Integer paise ensures accurate calculations, and object-oriented programming makes the system easier to maintain and extend.

Overall assignment conclusion

The Namma Metro Smart-Card Fare System should use C++ object-oriented programming with:

- `long long` for money in paise.
- `constexpr` for fixed fare values.
- `enum class` for zones.
- `switch` or an indexed fare table for fare selection.
- `if-else` for peak-hour and balance checking.
- `SmartCard` and `MonthlyPass` classes.
- Input validation to prevent invalid transactions.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Object-Oriented Analysis	K5 – Evaluate	4
Task 2 – Tokens and Constants	K5 – Evaluate	3
Task 3 – Data Types and Compatibility	K5 – Evaluate	4
Task 4 – Implicit Conversions	K5 – Evaluate	4

Task 5 – Operators and Precedence	K5 – Evaluate	4
Task 6 – Control Structures	K5 – Evaluate	4
Task 7 – Procedural vs Object-Oriented	K5 – Evaluate	4
Task 8 – Input and Output	K5 – Evaluate	4
Task 9 – Implementation & Testing	K5 – Evaluate	4
Task 10 – Expectation vs Execution	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		42
Task 12 – Day-4 Design Challenge	K5 – Evaluate	8
TOTAL		50

Submission Requirements

- GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.